13 — Index Interview Guide: 50+ Questions with Detailed Answers

> *Complete preparation guide for PostgreSQL index interviews — from junior to staff engineer level.*

Table of Contents

How to Use This Guide

This guide is organized in increasing complexity. For:

- **Junior roles**: Master Q1–Q20 completely
- **Mid-level roles**: Q1–Q40, with production examples
- **Senior roles**: All questions, plus system design section
- **Staff/Principal**: All questions plus ability to discuss tradeoffs at architecture level

Practice verbalizing answers out loud. For technical interviews, always walk through your reasoning before giving the final answer.

Interview Level Framework

```
Junior (0-2 years):
  ✓ What is an index and why does it exist?
  ✓ What is a B-tree?
  ✓ When would an index not be used?
  ✓ What is a composite index leading column rule?

Mid-level (2-5 years):
  ✓ All junior topics
  ✓ GIN, GiST, BRIN tradeoffs
  ✓ Covering indexes and index-only scans
  ✓ EXPLAIN output analysis
  ✓ Partial and expression indexes
  ✓ Index maintenance basics
```

```
Senior (5+ years):
  ✓ All mid-level topics
  ✓ Index bloat diagnosis and remediation
  ✓ Planner cost model deep dive
  ✓ Concurrent DDL operations
  ✓ Production incident handling
  ✓ Index strategy for large-scale systems
```

Fundamentals (Q1-Q12)

Q1. What is a database index? What problem does it solve?

Expected answer (1-2 minutes):

An index is a separate data structure that maps column values to the heap row locations (ctids) where those rows are stored. It solves the **sequential scan problem**: without an index, finding rows matching a predicate requires reading every page of the table (O(n) I/O). An index allows the database to find matching rows in O(log n) time for a B-tree, reducing thousands of page reads to 3–5 page reads.

Real-world framing: An index is like the index at the back of a textbook. Instead of reading the entire book to find "B-tree", you look up "B-tree" in the index which tells you exactly which pages to turn to.

Follow-up: "What's the cost of adding an index?"

- Storage: the index occupies disk space (30-70% of the indexed data for B-tree)
- Write overhead: every INSERT/UPDATE/DELETE on indexed columns must update the index
- Planner cost: more indexes = more plan options to evaluate (usually negligible)

Q2. Why is a sequential scan sometimes faster than an index scan?

Key insight: This tests whether you understand the cost model, not just index benefits.

When a query returns a large fraction of rows (high selectivity), the index scan performs many random I/Os — one for each matching heap page. Random I/O is much more expensive than sequential I/O, especially on traditional hard drives. At some threshold (typically 1-5% selectivity), it's cheaper to read the table sequentially.

Example: `SELECT * FROM orders WHERE status = 'completed'` — if 80% of orders are completed, an index scan would require 80% of heap pages to be read in random order. A sequential scan reads 100% of pages but in order, using OS read-ahead. The sequential scan wins.

The crossover point depends on `random_page_cost` / `seq_page_cost`. For SSDs, set `random_page_cost = 1.1` to tell the planner random reads are cheap.

Q3. What is the query planner's cost model? How does it decide to use an index?

The planner assigns numeric costs to each possible plan:

- `seq_page_cost = 1.0` (baseline)
- `random_page_cost = 4.0` (HDD default; use 1.1 for SSD)
- `cpu_tuple_cost = 0.01`

For each plan, it calculates:

```
Index Scan cost ≈ (tree_height × random_page_cost) + (matching_rows ×
random_page_cost) + (matching_rows × cpu_tuple_cost)
Seq Scan cost ≈ (total_pages × seq_page_cost) + (total_rows × cpu_tuple_cost)
```

The planner uses statistics from `pg_statistic` to estimate how many rows match (selectivity), then computes costs for both plans and chooses the cheapest.

Q4. How do you check if a query is using an index?

Use `EXPLAIN (ANALYZE, BUFFERS)`:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;
```

Look for:

- `Index Scan using idx_name` → index being used
- `Index Only Scan` → covering index, no heap access
- `Bitmap Index Scan` → multiple rows, collected and sorted before heap access
- `Seq Scan` → no index used

The `Buffers: shared hit=N read=M` line shows cache hits vs disk reads.

Q5. You created an index but the query still does a sequential scan. What are the possible reasons?

Seven common reasons:

1. **Low selectivity**: the query matches too many rows (>5-10%), making seq scan cheaper
2. **Function on column**: `WHERE UPPER(name) = ?` doesn't use index on `name`
3. **Type mismatch**: implicit cast prevents index usage
4. **Wrong column**: query uses a column not covered by the index
5. **Stale statistics**: `ANALYZE` not run after data changes; planner uses wrong estimates
6. **Wrong `random_page_cost`**: set too high (default 4.0 good for HDD, 1.1 for SSD)
7. **Small table**: seq scan is faster for very small tables regardless of index

Diagnostic steps:

```
ANALYZE table_name;   -- refresh statistics
SET random_page_cost = 1.1;  -- for SSD
EXPLAIN (ANALYZE, BUFFERS) query;  -- check actual vs estimated rows
```

Q6. What is index selectivity? How does it relate to when an index is useful?

Selectivity is the fraction of rows a predicate returns: `selectivity = matching_rows / total_rows`.

High selectivity (low fraction matching) = very useful index. Example: `WHERE user_id = 42` on 1M rows — 1/1,000,000 = 0.000001 selectivity → perfect for index.

Low selectivity (high fraction matching) = index not useful. Example: `WHERE status = 'active'` where 70% of rows are active → 0.7 selectivity → seq scan better.

The "crossover point" is approximately 1-5% selectivity, depending on `random_page_cost`:

- SSD (random_page_cost=1.1): crossover at ~30-50% — index used more aggressively

- HDD (random_page_cost=4.0): crossover at ~1-5% — index only for highly selective queries

---

Q7. What is a covering index? How does an index-only scan work?

A **covering index** includes all columns referenced by a query (both WHERE and SELECT columns). An **index-only scan** occurs when all needed data is in the index, so heap pages never need to be fetched.

```
-- Query: SELECT email, created_at FROM users WHERE user_id = 42;
-- Covering index:
CREATE INDEX ON users(user_id) INCLUDE (email, created_at);
-- Result: Index Only Scan — reads only 3-4 index pages, 0 heap pages
```

For index-only scans to work without heap fetches, the visibility map must confirm all heap pages are "all-visible" (set by VACUUM). Otherwise, heap pages are fetched for visibility checks.

Key benefit: 2-5× faster than regular index scan for lookup-heavy workloads.

Q8. What is a partial index? Give a real-world use case.

A partial index covers only a subset of rows — those matching a WHERE predicate at index creation time.

```
CREATE INDEX idx_tasks_pending ON tasks(created_at) WHERE status = 'pending';
```

Real-world use case: A task queue with 100M rows, but only 100K are 'pending'. Workers query pending tasks continuously. A full index on `status, created_at` would be ~1.6 GB. A partial index covering only 'pending' tasks would be ~1.6 MB — 1000× smaller, fitting entirely in cache.

For the query to use the index, the query's WHERE clause must imply the index predicate:

```
SELECT * FROM tasks WHERE status = 'pending' ORDER BY created_at LIMIT 10;
-- ✓ Uses partial index
```

Q9. What is a composite index? What is the leading column rule?

A composite index is built on multiple columns. Keys are sorted **lexicographically** — first by column 1, then by column 2 within each column 1 value, etc.

The **leading column rule**: a composite index on (a, b, c) can be used only for queries that include a filter on the leading column(s) as a prefix:

- Query on `a` alone: uses index (prefix of 1)
- Query on `a, b` : uses index (prefix of 2)
- Query on `b` alone: cannot use index (missing leading column)
- Query on `b, c` : cannot use index (missing leading column)

Exception: The SQL ORDER of predicates doesn't matter — the planner reorders them. `WHERE b=2 AND a=1` is the same as `WHERE a=1 AND b=2` — both use the index for (a, b).

Q10. Why should equality conditions come before range conditions in a composite index?

Once the index scan encounters a range condition, all subsequent columns can only be used as post-scan filters, not for narrowing the scan itself.

```
-- Good: equality first
CREATE INDEX ON orders(customer_id, created_at);
-- WHERE customer_id = 42 AND created_at > '2024-01-01'
-- Both columns used for scan: navigate to (42, '2024-01-01'), scan forward

-- Bad: range first
CREATE INDEX ON orders(created_at, customer_id);
-- WHERE customer_id = 42 AND created_at > '2024-01-01'
-- Only created_at used for scan; customer_id is a post-scan filter
-- Must scan all customers with recent orders, then filter for 42
```

Q11. What PostgreSQL index types exist and when would you use each?

Type	Use Case	Key Feature
B-tree (default)	Equality, range, sort	Sorted, general purpose
Hash	Equality only, long keys	O(1) lookup, smaller for long keys
GIN	Arrays, JSONB, full-text	Inverted index for multi-valued types
GiST	Geometry, ranges, exclusion	Bounding values, overlapping data
BRIN	Very large tables, sequential data	Tiny, block range summaries
SP-GiST	Points, IP prefixes, text prefixes	Non-overlapping space partitioning

Default choice: **B-tree**. Move to specialized types when:

- JSONB/arrays/full-text → GIN
- Geometry/ranges → GiST
- 100M+ row time-series → BRIN
- IP address subnet queries → SP-GiST

Q12. What is an expression index? When do you need one?

An expression index stores the result of a function applied to a column, enabling the planner to use it for queries that apply the same function.

You need one when:

1. Queries consistently apply a function before comparing: `WHERE lower(email) = ?`
2. A regular index on the raw column would not be used for that query pattern

```
CREATE INDEX ON users(lower(email));    -- enables: WHERE lower(email) =
'foo@bar.com'
CREATE INDEX ON events((data->>'type')); -- enables: WHERE data->>'type' = 'click'
CREATE INDEX ON orders(DATE(created_at)); -- enables: WHERE DATE(created_at) =
'2024-06-15'
```

The function must be IMMUTABLE (deterministic, no session state). `lower()`, `upper()`, `md5()`, `date_trunc(text, timestamp)` are immutable. `now()`, `current_user` are not.

B-Tree Deep Dive (Q13-Q22)

Q13. Describe the B-tree data structure used by PostgreSQL indexes.

A PostgreSQL B-tree (nbtree) consists of:

- **Root page**: entry point for all lookups
- **Internal pages**: contain separator keys + child page pointers; navigated to find leaf
- **Leaf pages**: contain actual index entries (key + ctid); linked in a doubly-linked list
- **Meta page** (page 0): stores tree metadata

Properties:

- Perfectly balanced: all leaf pages at same depth
- Each internal page has ~400 entries (for 4-byte int keys) → tree depth rarely exceeds 4
- Leaf pages doubly-linked for efficient range scans in both directions
- Keys within each page are stored in sorted order

For a 100M row table with 4-byte integer keys: tree height ≈ 4, meaning any lookup touches 4 pages regardless of which key is sought.

Q14. How does a range scan work in a B-tree index?

1. Navigate tree to the first key satisfying `key >= lower_bound` (same as equality lookup)
2. Arrive at the leaf page containing the lower_bound key
3. Scan forward through the leaf page, collecting ctids for matching entries
4. Follow the `next_page` pointer to the next leaf page and continue
5. Stop when a key exceeds the upper_bound or the scan exhausts leaf pages

This works because leaf pages are sorted and doubly-linked. The range scan reads contiguous leaf pages sequentially — efficient for both B-tree lookup and I/O.

Q15. What is a page split? How does PostgreSQL handle it?

A page split occurs when a new entry must be inserted into a full page. PostgreSQL:

1. Allocates a new page
2. Moves ~half the entries from the full page to the new page
3. Updates sibling links (doubly-linked list of leaf pages)
4. Inserts a new separator key into the parent internal page
5. If the parent is also full, cascades the split upward (rare)

Splits are expensive (multiple page writes, exclusive locks). They are minimized by:

- Setting fill factor to 70-80% (reserves space in each page for future inserts)
- Using sequential (monotonically increasing) keys (splits only occur at the "right end")

Q16. What is a HOT update and how does it relate to index design?

HOT (Heap Only Tuple) is an optimization where an UPDATE that modifies only non-indexed columns can skip updating the index. The new heap tuple points to the old tuple's ctid in the index (via a chain of pointers in the heap). The index entry still points to the old ctid, which redirects to the new tuple.

Implication for index design: Index as few frequently-updated columns as possible. Every index that covers an updated column breaks the HOT chain and requires writing to both heap and index.

Example: If `orders` has an index on `status` and you frequently update `status`, every update writes to the index. If the index was only on `customer_id` (rarely updated), updates to `status` are HOT-compatible.

Q17. What is index deduplication in PostgreSQL 13+ and how does it help?

Deduplication combines multiple leaf entries with the same key into a single "posting list" entry. For example, if 1000 orders have `customer_id = 42`, instead of 1000 separate leaf entries `[(42, ctid1), (42, ctid2), ..., (42, ctid1000)]`, there is one entry `[42, {ctid1, ctid2, ..., ctid1000}]`.

Benefits:

- Reduces index size by up to 90% for low-cardinality columns
- More entries fit per page → fewer pages to read → better cache utilization
- Especially valuable for status columns, boolean flags, category IDs

Enabled by default in PG13+. Can be disabled per-index with `deduplicate_items = off`.

Q18. When does a B-tree index support ORDER BY without an explicit sort?

When the index sort order matches the ORDER BY sort order exactly:

- Index `(customer_id ASC, created_at ASC)` supports `ORDER BY customer_id ASC, created_at ASC`
- Index `(customer_id ASC, created_at ASC)` also supports `ORDER BY customer_id DESC, created_at DESC` (reads backward)
- Index `(customer_id ASC, created_at ASC)` does NOT support `ORDER BY customer_id ASC, created_at DESC` (mixed directions)

For the last case, create: `CREATE INDEX ON orders(customer_id ASC, created_at DESC)`

Combined with `LIMIT`, an index that matches `ORDER BY` allows the planner to stop after finding exactly LIMIT rows — extremely efficient for pagination.

Q19. What is fill factor and when should you change it?

Fill factor controls how full each page is during index construction (and after rebuilds). Default is 90% (10% free space reserved for updates).

Lower fill factor (70-80%): Reserve more space → fewer page splits → better for tables with heavy UPDATEs to indexed columns. Write performance improves because more HOT updates are possible. Read performance is slightly worse (more pages).

Higher fill factor (95-100%): No reserved space → more entries per page → better read performance for read-heavy, mostly-INSERT workloads (like append-only event logs).

```
CREATE INDEX ON orders(customer_id) WITH (fillfactor = 70);
```

Q20. What is the difference between an Index Scan and a Bitmap Index Scan?

Index Scan: Fetches heap tuples immediately for each index entry, in index order (random order relative to the heap). Used when few rows match. For each ctid, a heap page is read — potentially many random reads.

Bitmap Index Scan + Bitmap Heap Scan: Collects ALL matching ctids from the index first, creates a bitmap, sorts ctids by heap page number, then reads heap pages in order. Used when many rows match. More efficient than Index Scan when many heap pages must be read — because it reads them sequentially (sorted), not randomly.

Transition from Index Scan → Bitmap Index Scan happens around 10-100 matching rows depending on page size and `effective_cache_size`.

Q21. Can a single query use multiple indexes? How?

Yes, via **Bitmap AND / OR** operations:

```
-- Two separate indexes:
CREATE INDEX ON orders(customer_id);
CREATE INDEX ON orders(status);

-- Query: WHERE customer_id = 42 AND status = 'pending'
-- Plan might be:
--   Bitmap Index Scan on idx_customer → set of ctids
--   Bitmap Index Scan on idx_status → set of ctids
--   BitmapAnd → intersection
--   Bitmap Heap Scan
```

OR combination:

```
-- WHERE customer_id = 42 OR status = 'pending'
--   Bitmap Index Scan on idx_customer → set A
--   Bitmap Index Scan on idx_status → set B
--   BitmapOr → union
--   Bitmap Heap Scan
```

This is why multiple single-column indexes can sometimes substitute for a composite index, though a properly designed composite index is usually more efficient.

Q22. What is index-only scan and what conditions must be met for it to occur?

An index-only scan (IOS) returns data entirely from the index without reading heap pages. Required conditions:

1. All columns referenced in SELECT and WHERE are in the index (as key or INCLUDE columns)
2. The heap page is marked "all-visible" in the visibility map

If condition 2 fails, PostgreSQL fetches the heap page for a visibility check (a "heap fetch"). After VACUUM marks pages as all-visible, heap fetches drop to 0.

EXPLAIN shows: `Heap Fetches: 0` for a true IOS. A non-zero value indicates either recent modifications or missing VACUUM.

Specialized Index Types (Q23-Q30)

Q23. When would you choose GIN over B-tree for text search?

B-tree for text supports only:

- Equality (`= 'exact string'`)
- Prefix range (`LIKE 'prefix%'`)

GIN is needed for:

- Full-text search: `WHERE search_vector @@ to_tsquery('english', 'word1 & word2')`
- Substring search (with pg_trgm): `WHERE name LIKE '%middle%'`
- Array containment: `WHERE tags @> ARRAY['tag1', 'tag2']`
- JSONB containment: `WHERE data @> '{"field": "value"}'`

GIN is an inverted index: each word/element maps to all rows containing it. For full-text search, GIN is preferred over GiST because it's exact (no false positives) and faster for query execution, though larger and slower to build/update.

Q24. What is a BRIN index and when should you use it?

BRIN (Block Range INdex) stores min/max summaries for ranges of heap pages (typically 128 pages = 1 MB). It's useful when:

1. The table is very large (100M+ rows)
2. The indexed column has high physical correlation with row insertion order
3. Queries access date ranges or sequential ID ranges

Example: An `events` table with 10B rows, `created_at` timestamp (naturally sequential since rows are inserted in time order). A B-tree index would be ~160 GB. A BRIN index is ~30 MB (5000× smaller) and still prunes most of the table for time-range queries.

Not suitable for: Random values (UUIDs, shuffled data), point queries requiring exact lookups, columns with low correlation.

Q25. What is the difference between jsonb_ops and jsonb_path_ops for GIN indexes on JSONB?

`jsonb_ops` (default): Indexes every key, value, and key-value pair. Supports `@>` , `?` , `?|` , `?&` , `@@` , `@?` operators.

`jsonb_path_ops` : Indexes only values (not keys separately). Supports only `@>` . Produces ~25-30% smaller index with slightly faster `@>` queries.

Choose `jsonb_path_ops` when:

- Only containment queries (`@>`) are used
- Storage or query speed for `@>` is the priority

Choose `jsonb_ops` when:

- Key existence checks (`data ? 'field'`) are needed
- Multiple operators are used (`@@` , `@?` for jsonpath)

Q26. How does a GiST exclusion constraint work for preventing overlapping bookings?

```
CREATE TABLE bookings (
    room_id INTEGER,
    period TSTZRANGE,
```

```
    EXCLUDE USING GIST (room_id WITH =, period WITH &&)
);
```

The exclusion constraint uses GiST to enforce that no two rows in the table satisfy both conditions simultaneously: same `room_id` (using `=`) AND overlapping `period` (using `&&`).

When inserting a new booking, PostgreSQL uses the GiST index to efficiently check for conflicting existing bookings (same room, overlapping time). If found, the insert fails. This is atomic and handles concurrent inserts correctly via locking.

Without this, you'd need application-level serialization with table locks — much harder to implement correctly.

Q27. What is SP-GiST and how does it differ from GiST?

SP-GiST (Space-Partitioned GiST) uses **non-overlapping** space partitioning:

- Each level divides space into non-overlapping regions (quadrants for 2D, prefixes for text)
- A data item belongs to exactly one partition at each level
- No false positives in lookups (unlike GiST bounding boxes which can overlap)

GiST uses **overlapping bounding boxes**:

- Internal nodes store bounding boxes that may overlap
- False positives possible (bounding boxes overlap query region but actual data doesn't)
- Recheck step eliminates false positives at the heap level

SP-GiST is better for uniformly distributed data (points spread across space). GiST handles clustered data and overlap queries better. GiST supports exclusion constraints; SP-GiST does not.

Q28. What is the pending list in GIN and when would you disable fastupdate?

GIN maintains a **pending list**: a small WAL-logged buffer of newly-inserted entries that haven't been merged into the main GIN B-tree yet. This avoids expensive per-entry insertions into the GIN structure on every write.

Queries check both the main GIN tree and the pending list, merging results.

Disable `fastupdate (WITH (fastupdate = off))` when:

- Query latency consistency matters and the pending list adds variable cost
- The table is read-heavy with few inserts (pending list rarely used)
- Autovacuum is poorly configured and the list grows very large

Enable (default) when:

- Insert throughput matters (bulk inserts, event streams)
- GIN is on a column with many values per row (many pending entries per insert)

Q29. How does the correlation statistic in pg_stats affect BRIN effectiveness?

`pg_stats.correlation` measures how well the column's logical sort order matches the physical heap order (1.0 = perfect, 0.0 = random, -1.0 = reversed).

For BRIN, high correlation (>0.8) means:

- Each block range's [min, max] summary is narrow and non-overlapping with adjacent ranges
- A query for a specific date range matches only a few block ranges

- BRIN efficiently skips most of the table

Low correlation (near 0) means:

- Each block range's [min, max] spans nearly the entire value domain
- Almost every block range "might" contain matches
- BRIN provides no benefit — every block range is scanned

Always check `pg_stats.correlation` before creating BRIN. If < 0.8, use B-tree.

Q30. When would you use a hash index over a B-tree?

Hash indexes store a fixed-size hash (4 bytes) of the key rather than the full key. This makes them smaller and faster for equality lookups on long keys.

Use hash when ALL of the following are true:

1. Key is long (VARCHAR(100+), long TEXT, URLs, SHA hashes)
2. All queries are strict equality (`=`) — no ranges, no ORDER BY, no LIKE
3. No UNIQUE constraint needed (hash doesn't support unique)

For integer or short text columns, B-tree is essentially identical in performance but more versatile. Note: before PostgreSQL 10, hash indexes were NOT WAL-logged and unsafe for production. After PG10, they are fully safe.

Index Design & Strategy (Q31-Q40)

Q31. How would you design indexes for a high-traffic REST API endpoint?

Step-by-step approach:

1. **Identify the query pattern**: `WHERE tenant_id = ? AND status = 'active' ORDER BY created_at DESC LIMIT 10`
2. **Identify equality vs range columns**: `tenant_id` = equality, `status` = equality, `created_at` = sort
3. **Order columns**: equality first, sort last
4. **Check returned columns**: if returning `id, total`, add INCLUDE
5. **Consider partial**: if most rows are 'completed', partial index on 'active' saves space

```
CREATE INDEX CONCURRENTLY idx_orders_api
ON orders(tenant_id, status, created_at DESC)
INCLUDE (id, total)
WHERE status IN ('pending', 'processing', 'active');
```

6. **Verify with EXPLAIN**: confirm Index Only Scan, 0 heap fetches
7. **Monitor**: check idx_scan in pg_stat_user_indexes after deployment

---

Q32. How many indexes should a table have?

There is no fixed rule, but guidelines:

- **Primary key**: always (PostgreSQL creates automatically)
- **Foreign keys**: always index them (PostgreSQL does NOT auto-create FK indexes)

- **Frequently queried columns**: add indexes for columns in common WHERE clauses
- **Unique constraints**: implicit unique index

Danger zones:

- More than 5-8 indexes on a write-heavy table → investigate write performance
- Any index with `idx_scan = 0` after 30+ days → candidate for removal

The right question: "What is the write/read ratio?" High write tables should minimize indexes. High read tables can afford more.

Q33. How do you decide between a composite index and multiple single-column indexes?

Use **composite** when:

- Queries always (or usually) filter on both columns together
- The combination provides better selectivity than either column alone
- You need ORDER BY on multiple columns (composite can eliminate sort)
- You want index-only scans covering multiple columns

Use **multiple single-column** when:

- Query patterns vary: sometimes col A alone, sometimes col B alone, sometimes both
- The planner's BitmapAnd is effective for the combined case (each index is highly selective)
- Adding a column to an existing composite would make it too wide

Note: A composite index on (a, b) already handles `WHERE a = ?` queries efficiently, so a separate index on just (a) is redundant.

Q34. How should you handle indexes on a table that receives millions of inserts per day?

For write-heavy tables:

1. **Minimize index count**: only index columns used in WHERE/JOIN for the most critical queries
2. **Use partial indexes**: only index the "hot" working set (e.g., last 30 days, status='pending')
3. **Defer index creation**: load data first (disable autovacuum), then create indexes with maintenance_work_mem
4. **Consider BRIN for time-series**: huge tables with sequential data → BRIN is tiny and near-zero write overhead
5. **Use fill factor 70-80%**: reduces page splits for UPDATE-heavy indexes
6. **Monitor pg_stat_user_tables.n_live_tup vs n_dead_tup**: tune autovacuum
7. **Use pg_stat_statements** to confirm which queries actually need the index

---

Q35. Why should you always index foreign key columns in PostgreSQL?

Unlike some databases (MySQL), PostgreSQL does NOT automatically create indexes on foreign key columns.

Without FK indexes, two problems occur:

1. **Referential integrity checks**: when a referenced parent row is deleted or updated, PostgreSQL must find all child rows to enforce the constraint. Without an index, this is a sequential scan of the child table — for every parent deletion.
2. **JOIN performance**: `orders JOIN customers ON orders.customer_id = customers.id` performs a sequential scan on `orders` without an index on `customer_id`.

For a table with 1M orders and 10K customers, deleting a customer without an FK index triggers a 1M-row sequential scan. With an index, it's a 3-page lookup.

```
-- Find FK columns without indexes:
SELECT
    tc.table_name, kcu.column_name
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu USING (constraint_name)
WHERE tc.constraint_type = 'FOREIGN KEY'
  AND NOT EXISTS (
      SELECT 1 FROM pg_indexes
      WHERE tablename = tc.table_name
        AND indexdef LIKE '%' || kcu.column_name || '%'
  );
```

Q36. How do you design indexes for a soft-delete pattern?

Soft-delete: rows are marked deleted with `deleted_at TIMESTAMPTZ` instead of physically deleted.

Problems:

- Queries almost always filter for non-deleted rows: `WHERE deleted_at IS NULL`
- Standard indexes contain deleted rows, adding noise and size

Solution: **Partial indexes** excluding deleted rows:

```
-- Bad: full index includes deleted rows
CREATE INDEX ON users(email);
-- 30% of users are deleted → 30% of index is "dead weight"

-- Good: partial index excludes deleted rows
CREATE INDEX ON users(email) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX ON users(email) WHERE deleted_at IS NULL;  -- unique active
emails

-- Query must include the predicate to use the index:
SELECT * FROM users WHERE email = 'foo@bar.com' AND deleted_at IS NULL;
```

Q37. How do you handle pagination with indexes?

Wrong approach (OFFSET pagination): `LIMIT 20 OFFSET 10000` — forces reading and discarding 10,000 rows even with an index.

Correct approach (keyset/cursor pagination):

```
-- Index: (created_at, id)
-- Page 1:
SELECT id, created_at, title FROM posts ORDER BY created_at DESC, id DESC LIMIT 20;

-- Page 2 (cursor = last row's created_at and id):
SELECT id, created_at, title FROM posts
```

```
WHERE (created_at, id) < ('2024-06-15 10:00:00', 12345)
ORDER BY created_at DESC, id DESC
LIMIT 20;
-- This uses the index efficiently — jumps directly to the cursor position
```

The composite index on (created_at DESC, id DESC) serves both the ORDER BY and the WHERE clause for pagination, making each page fetch O(log n) regardless of page number.

Q38. You have a JSONB column. When would you use a GIN index vs expression indexes?

GIN index (`CREATE INDEX ON t USING GIN (data)`):

- When queries use containment (`@>`) or key existence (`?`)
- When multiple fields are queried in different combinations
- When queries use jsonpath operators (`@@` , `@?`)
- More flexible but larger

Expression index (`CREATE INDEX ON t ((data->>'field'))`):

- When ONE specific field is queried by equality very frequently
- When the field has high cardinality (many distinct values)
- Smaller and faster for that specific field's equality queries
- Must match the exact expression in the query

Combined strategy:

```
-- Frequently used field: expression index (small, fast, specific)
CREATE INDEX ON events((data->>'user_id')::BIGINT);

-- General multi-field queries: GIN index
CREATE INDEX ON events USING GIN (data jsonb_path_ops);
```

Q39. How do you handle case-insensitive unique constraints?

Standard UNIQUE constraint is case-sensitive. For case-insensitive uniqueness:

Option 1: Expression unique index

```
CREATE UNIQUE INDEX ON users(lower(email));
-- Prevents 'alice@example.com' and 'Alice@Example.com' from coexisting
-- Query must use: WHERE lower(email) = lower($1)
```

Option 2: citext extension

```
CREATE EXTENSION IF NOT EXISTS citext;
ALTER TABLE users ALTER COLUMN email TYPE citext;
CREATE UNIQUE INDEX ON users(email);  -- now case-insensitive
-- Queries: WHERE email = 'Alice@Example.com'  -- works naturally
```

Option 3: Normalize before insert (application layer)

- Always store emails in lowercase: `INSERT INTO users(email) VALUES (lower($1))`

- Standard UNIQUE index works

Option 1 (expression index) is most portable and doesn't require schema changes. Option 2 (citext) is cleanest at the application layer.

Q40. How would you design indexes for a multi-tenant SaaS application?

Key insight: `tenant_id` should be the leading column of EVERY composite index, because every query filters by tenant.

```
-- Pattern: every index starts with tenant_id
CREATE INDEX ON orders(tenant_id, status, created_at DESC);
CREATE INDEX ON users(tenant_id, email);
CREATE INDEX ON products(tenant_id, category_id, price);

-- For full-text search per tenant:
CREATE INDEX ON articles(tenant_id) INCLUDE (title, search_vector);
-- Plus GIN on search_vector (separate, not tenant-prefixed — but query filters by
tenant_id first)

-- Partial indexes per tenant for very active tenants (optional):
CREATE INDEX ON orders(created_at DESC) WHERE tenant_id = 42;
-- Only viable if this tenant is extraordinarily large and active
```

This ensures the planner always uses the index for tenant isolation, and the index scan immediately narrows to the specific tenant's rows, which is then further filtered.

Query Optimization with Indexes (Q41-Q48)

Q41. Walk me through reading this EXPLAIN output:

```
Index Scan using idx_orders_customer on orders  (cost=0.56..8.58 rows=5 width=48)
                                                  (actual time=0.023..0.031 rows=5
loops=1)
  Index Cond: (customer_id = 42)
  Buffers: shared hit=4
```

Answer breakdown:

- `Index Scan using idx_orders_customer` : B-tree (or hash) index scan
- `cost=0.56..8.58` : startup cost..total cost (in abstract planner units, seq_page_cost=1.0)
  - 0.56 = cost to start returning rows (tree navigation)
  - 8.58 = cost when last row is returned
- `rows=5` : planner estimated 5 rows would match
- `width=48` : average row width in bytes (for memory estimation)
- `actual time=0.023..0.031` : actual wall clock time in ms (startup..total)
- `rows=5` : actual rows returned — matches estimate, good statistics!
- `loops=1` : this node was executed once
- `Buffers: shared hit=4` : 4 pages found in shared_buffers (no disk I/O)

Green flags: estimated matches actual rows, low actual time, shared hits > reads. **Red flags**: actual rows >> estimated rows (stale stats), shared read >> hit (cache miss).

Q42. What does "Rows Removed by Filter" mean in EXPLAIN and why does it matter?

"Rows Removed by Filter" appears when the planner reads rows from an index but then discards them because they don't satisfy a non-indexed condition.

```
Index Scan using idx_orders_customer on orders
  Index Cond: (customer_id = 42)
  Filter: (status = 'active')
  Rows Removed by Filter: 47
```

This means: 47+5 = 52 rows were fetched from the heap (for customer 42), but 47 were discarded because status != 'active'. Only 5 rows were returned.

Why it matters: Each discarded row required a heap page read. If the ratio is high (many removed vs returned), consider adding `status` to the composite index to avoid fetching those heap pages.

Fix: Add `status` to the index or create a partial index:

```sql
CREATE INDEX ON orders(customer_id, status);  -- or
CREATE INDEX ON orders(customer_id) WHERE status = 'active';
```

Q43. What causes "actual rows" to be very different from "rows" in EXPLAIN ANALYZE?

This mismatch indicates **stale or inaccurate statistics** in `pg_statistic`.

Causes:

1. **Table not analyzed after data changes**: run `ANALYZE table_name`
2. **Statistics target too low**: increase `default_statistics_target` or per-column target
3. **Correlated columns**: planner assumes column independence (use extended statistics)
4. **Data skew**: most values are common, a few are rare (n_distinct estimate wrong)

```sql
-- Fix: update statistics
ANALYZE orders;

-- If that doesn't help: increase statistics target for this column
ALTER TABLE orders ALTER COLUMN customer_id SET STATISTICS 500;
ANALYZE orders;

-- For correlated columns (e.g., city + state always go together):
CREATE STATISTICS stats_city_state ON city, state FROM addresses;
ANALYZE addresses;
```

Q44. When would the planner use a Bitmap Index Scan instead of an Index Scan?

The planner chooses Bitmap Index Scan (BIS) over Index Scan when:

1. **Many rows match**: too many for efficient random Index Scan, too few for Seq Scan
2. **Multiple indexes are combined**: BitmapAnd/BitmapOr of multiple indexes

3. **The index is lossy** (e.g., BRIN): always uses BIS

Bitmap Index Scan works by:

1. Reading ALL matching ctids from the index
2. Creating a bitmap sorted by heap page number
3. Bitmap Heap Scan reads heap pages in order (improving sequential I/O)

This trades extra memory (for the bitmap) for better I/O patterns (sequential heap reads instead of random). Controlled by `work_mem` — a larger bitmap fits more ctids.

Q45. How does LIMIT affect index usage?

`LIMIT` can dramatically change the planner's decision:

Without LIMIT: the planner estimates total cost. For a query with high selectivity, an index scan might be cheaper even for many rows.

With LIMIT N: the planner can use a **startup cost** comparison. If the index provides results in the correct ORDER BY order, the planner can stop after N rows. This makes index scans much cheaper for `LIMIT 10` even on large tables.

```
-- Without LIMIT: planner may choose Seq Scan + Sort (cheaper total cost)
EXPLAIN SELECT * FROM orders WHERE tenant_id = 1 ORDER BY created_at DESC;

-- With LIMIT 10: planner chooses Index Scan (stops after 10 rows in order)
EXPLAIN SELECT * FROM orders WHERE tenant_id = 1 ORDER BY created_at DESC LIMIT 10;
```

Create indexes that match your ORDER BY + LIMIT patterns for best performance.

Q46. What is the difference between Index Cond and Filter in EXPLAIN?

Index Cond: conditions evaluated BY the index — these narrow which index entries are traversed. Only entries satisfying the Index Cond are read from the index. No heap fetch for entries that don't match Index Cond.

Filter: conditions evaluated AFTER fetching from the index but BEFORE returning rows. Heap pages are fetched for all entries passing Index Cond, then Filter is applied.

```
Index Scan using idx_orders_cust_date on orders
  Index Cond: (customer_id = 42) AND (created_at > '2024-01-01')  ← index-level filter
  Filter: (total_amount > 100)                                     ← heap-level filter
  Rows Removed by Filter: 15
```

Index Cond: Only rows for customer 42 after Jan 2024 are read from the index. Filter: Heap rows are fetched and then filtered for total_amount > 100.

To convert a Filter into an Index Cond, add the column to the index.

Q47. How does `effective_cache_size` affect index usage?

`effective_cache_size` tells the planner the total amount of memory available for caching (shared_buffers + OS page cache). A higher value means the planner assumes more index and heap pages are cached in memory, reducing the perceived cost of random I/O.

Effect: higher `effective_cache_size` → lower cost for index scans → planner chooses index scans more aggressively.

Set it to ~75% of total RAM:

```
ALTER SYSTEM SET effective_cache_size = '24GB';  -- on a 32 GB server
SELECT pg_reload_conf();
```

If left at the default (4 GB) on a 32 GB server, the planner significantly underestimates how much data is cached and may prefer sequential scans over index scans unnecessarily.

Q48. What is a correlated subquery problem and how do indexes help?

N+1 query problem: instead of one JOIN, the application issues one query per row:

```
-- BAD: N+1 — one query per order
SELECT id FROM orders WHERE customer_id = 1;
-- Then for each order:
SELECT * FROM order_items WHERE order_id = $1;
-- 1000 orders → 1001 queries

-- GOOD: one JOIN
SELECT o.id, oi.*
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
WHERE o.customer_id = 1;
```

Indexes help N+1 by making each per-row lookup fast (index on `order_id`), but the correct fix is to eliminate N+1 entirely. An index on `order_items.order_id` is still needed for the JOIN to be efficient.

Production & Maintenance (Q49-Q56)

Q49. How do you add an index to a large production table without downtime?

Use `CREATE INDEX CONCURRENTLY`:

```
CREATE INDEX CONCURRENTLY idx_orders_customer ON orders(customer_id);
```

This builds the index in multiple passes while allowing concurrent reads AND writes. The table is never exclusively locked. Caveats: takes 2-3× longer than regular CREATE INDEX; if it fails (server crash, deadlock), it leaves an INVALID index that must be dropped.

For extremely large tables (>100M rows), also:

- Run during off-peak hours (still consumes I/O)
- Increase `maintenance_work_mem` for faster sort phase
- Monitor `pg_stat_progress_create_index` for progress

Q50. You notice a 3 GB index on a table that is 500 MB. What do you check and do?

1. Check for bloat: `CREATE EXTENSION pgstattuple; SELECT avg_leaf_density FROM pgstatindex('idx_name');`
2. If avg_leaf_density < 50%, severe bloat: `REINDEX INDEX CONCURRENTLY idx_name`
3. Check if the index is actually used: `SELECT idx_scan FROM pg_stat_user_indexes WHERE indexrelname = 'idx_name'`
4. Check if it's a composite index covering many columns (legitimately large)
5. Check if it's a GIN full-text index (can legitimately be larger than the table)
6. After REINDEX, tune autovacuum to prevent recurrence

Q51. An index exists on a column but queries are still slow. Walk me through your debugging process.

```
-- Step 1: Check if index is being used
EXPLAIN (ANALYZE, BUFFERS) slow_query;
-- Look for: Index Scan? Seq Scan?

-- Step 2: If Seq Scan — why isn't index used?
-- Check statistics
ANALYZE table_name;
EXPLAIN slow_query;  -- re-check

-- Step 3: Check random_page_cost (on SSD should be 1.1)
SHOW random_page_cost;
SET random_page_cost = 1.1;
EXPLAIN slow_query;

-- Step 4: Check if function on column breaks index
-- WHERE lower(email) = ? but index on raw email?

-- Step 5: If Index Scan is slow — check actual vs estimated rows
-- Large discrepancy → statistics issue
-- Many Rows Removed by Filter → index not covering right columns

-- Step 6: Check index bloat
SELECT * FROM pgstatindex('idx_name');
-- avg_leaf_density < 60% → REINDEX CONCURRENTLY

-- Step 7: Check if index-only scan is possible
-- Add INCLUDE columns, run VACUUM, check Heap Fetches
```

Q52. How do you monitor index health in production?

Key metrics to monitor (ideally via automated alerting):

```
-- 1. Unused indexes (run weekly)
SELECT indexname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes WHERE idx_scan = 0 AND NOT indisunique;

-- 2. Tables with high dead tuple ratio (needs vacuum tuning)
SELECT relname, n_dead_tup, n_live_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct
```

```
FROM pg_stat_user_tables WHERE n_dead_tup > 10000 ORDER BY dead_pct DESC;

-- 3. Invalid indexes (immediate action needed)
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;

-- 4. Index vs table size ratios (bloat signal)
SELECT indexrelname,
       ROUND(100.0 * pg_relation_size(indexrelid) / pg_relation_size(indrelid), 1)
AS pct
FROM pg_stat_user_indexes JOIN pg_index USING (indexrelid)
WHERE pg_relation_size(indrelid) > 0 ORDER BY pct DESC;
```

Q53. What is the impact of autovacuum configuration on index performance?

Autovacuum removes dead index entries, preventing bloat. Poor configuration leads to:

- Dead tuples accumulate → index bloat → slower queries (more pages to read)
- Visibility map not updated → index-only scans require heap fetches
- Statistics not refreshed → planner makes poor index choices

For high-churn tables, tune aggressively:

```
ALTER TABLE high_churn_table SET (
    autovacuum_vacuum_scale_factor = 0.05,    -- default 0.2
    autovacuum_vacuum_threshold = 1000,        -- default 50
    autovacuum_analyze_scale_factor = 0.02     -- default 0.1
);
```

Q54. How do you safely drop an index in production?

```
-- Step 1: Verify it's not enforcing a constraint
SELECT conname FROM pg_constraint WHERE conindid = 'idx_to_drop'::regclass::oid;

-- Step 2: "Soft drop" — rename first to verify no queries break
ALTER INDEX idx_to_drop RENAME TO _zz_unused_idx_to_drop_20240615;
-- Monitor for 24-48 hours. If no query errors: proceed.

-- Step 3: Drop concurrently (non-blocking)
DROP INDEX CONCURRENTLY _zz_unused_idx_to_drop_20240615;
-- Never use: DROP INDEX idx_name; -- this takes an exclusive lock!
```

Q55. What is the effect of creating too many indexes on write performance?

Each index on a table means:

- Every INSERT must write one additional index entry per index
- Every UPDATE that changes indexed columns must update those indexes
- Every DELETE must mark index entries as dead

In practice, each additional index on a write-heavy table reduces INSERT throughput by 5-15% depending on index size and caching. For a table with 15 indexes, inserts might be 50-75% slower than with 3-4 carefully

chosen indexes.

Measurement:

```
-- Check write stats
SELECT n_tup_ins, n_tup_upd, n_tup_del FROM pg_stat_user_tables WHERE relname =
'orders';
-- Check how many indexes the table has
SELECT count(*) FROM pg_indexes WHERE tablename = 'orders';
```

Q56. How would you migrate from an unindexed table to a properly indexed one without downtime?

```
-- 1. Identify necessary indexes from slow query log
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- 2. Create indexes concurrently (one at a time to avoid resource contention)
CREATE INDEX CONCURRENTLY idx1 ON orders(customer_id);
-- Wait for completion, verify no issues

CREATE INDEX CONCURRENTLY idx2 ON orders(created_at DESC);
-- Wait...

-- 3. Verify each index is used
EXPLAIN (ANALYZE) common_query1;
EXPLAIN (ANALYZE) common_query2;

-- 4. Monitor performance after each index
SELECT * FROM pg_stat_statements WHERE query LIKE '%orders%';

-- 5. Update statistics
ANALYZE orders;
```

System Design Questions (Q57-Q60)

Q57. Design an indexing strategy for a time-series database with 10 billion rows.

Requirements analysis:

- 10B rows, ~1 TB of data
- Write-heavy: millions of rows/day appended
- Query patterns: time-range queries, per-sensor queries, aggregations

Strategy:

1. **Partition by month**: 10B rows ÷ 36 months ≈ 278M rows/partition
2. **BRIN on timestamp per partition**: each partition has ~~278M sequential rows → BRIN tiny (~~5 MB each) and effective
3. **B-tree on (sensor_id, recorded_at) per partition**: for per-sensor time-range queries
4. **For aggregations**: consider materialized views refreshed incrementally

5. **Autovacuum tuning**: high write rate → aggressive autovacuum settings

```
CREATE TABLE sensor_data (
    sensor_id BIGINT,
    recorded_at TIMESTAMPTZ,
    value FLOAT
) PARTITION BY RANGE (recorded_at);

-- Per partition:
CREATE INDEX ON sensor_data_2024_06 USING BRIN (recorded_at) WITH
(autosummarize=on);
CREATE INDEX ON sensor_data_2024_06 (sensor_id, recorded_at);
```

Q58. Design an index strategy for a document search system with JSONB metadata and full-text content.

```
CREATE TABLE documents (
    id BIGSERIAL PRIMARY KEY,
    tenant_id BIGINT NOT NULL,
    type TEXT NOT NULL,                    -- 'contract', 'invoice', etc.
    content TEXT,
    metadata JSONB,
    search_vector TSVECTOR,               -- pre-computed
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Indexes:
-- 1. Multi-tenant prefix (always filter by tenant_id first)
CREATE INDEX ON documents(tenant_id, type, created_at DESC);

-- 2. Full-text search (per-tenant)
CREATE INDEX ON documents USING GIN (search_vector);

-- 3. JSONB metadata for specific common fields
CREATE INDEX ON documents((metadata->>'vendor'));
CREATE INDEX ON documents USING GIN (metadata jsonb_path_ops);

-- 4. Recent documents for dashboard
CREATE INDEX ON documents(tenant_id, created_at DESC)
INCLUDE (id, type, (metadata->>'title'))
WHERE created_at > '2024-01-01';

-- Trigger to maintain search_vector:
CREATE FUNCTION update_search_vector() RETURNS trigger AS $$
BEGIN NEW.search_vector = to_tsvector('english', COALESCE(NEW.content, '')); RETURN
NEW; END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trig_doc_fts BEFORE INSERT OR UPDATE ON documents FOR EACH ROW
EXECUTE FUNCTION update_search_vector();
```

Q59. How would you handle a situation where a critical query suddenly became slow after a data migration?

Systematic approach:

1. `EXPLAIN (ANALYZE, BUFFERS)` — compare new plan vs expected plan
2. Check if indexes still exist: `\d+ table_name`
3. Check if statistics are up to date: `ANALYZE table_name` → re-EXPLAIN
4. Check data volume changes: did the table grow significantly?
5. Check planner configuration: `random_page_cost`, `effective_cache_size`
6. Compare estimated vs actual rows: stale statistics?
7. Check if the migration changed data distribution (e.g., column values shifted)
8. Check for index bloat if migration included many DELETEs/UPDATEs

Quick fixes:

- `ANALYZE table_name` (most common fix for post-migration slowdowns)
- `REINDEX INDEX CONCURRENTLY` if bloat suspected
- Adjust `default_statistics_target` if data is very skewed

Q60. What tradeoffs would you consider when deciding whether to add an index to a 500 GB table in production?

Benefits:

- Query performance improvement (quantify: how much? for how many queries/second?)
- Reduction in CPU and I/O load for read queries

Costs:

- Storage: estimate new index size (`rows × entry_size × 1.2 / 0.9 / 8192` pages)
- Write overhead: % slowdown on INSERT/UPDATE/DELETE (run benchmark)
- Build time: for CONCURRENT build, how long? What I/O impact during build?
- Maintenance: adds to ongoing VACUUM and REINDEX maintenance burden

Questions to answer:

- How many queries/second benefit? What is their current P95 latency?
- What is the write:read ratio?
- Is the table already bloated? REINDEX CONCURRENTLY needed first?
- Can the benefit be achieved through query rewriting instead?
- Is there a time window for the concurrent build (high I/O during build)?

Decision framework:

```
Expected QPS × (old_latency - new_latency) × 0.001  > (Write_overhead_ms × Write_QPS × 0.001)
```

If read savings > write overhead → add the index.

Whiteboard/Coding Exercises

Exercise 1 (15 min): Index design from query patterns

Given table `events(id, user_id, event_type, properties JSONB, occurred_at)` and queries:

```
-- Q1: WHERE user_id = ? ORDER BY occurred_at DESC LIMIT 10
-- Q2: WHERE event_type = 'purchase' AND occurred_at > NOW() - INTERVAL '30 days'
-- Q3: WHERE properties @> '{"source": "mobile"}'
-- Q4: WHERE user_id = ? AND event_type = 'purchase'
```

Design the minimum set of indexes to efficiently support all four queries.

Solution:

```
CREATE INDEX ON events(user_id, occurred_at DESC);      -- Q1, Q4 (partial)
CREATE INDEX ON events(event_type, occurred_at DESC);    -- Q2
CREATE INDEX ON events USING GIN (properties);           -- Q3
-- Q4: uses first index (equality on user_id, then filter on event_type)
-- or add: CREATE INDEX ON events(user_id, event_type, occurred_at DESC);
```

Exercise 2 (10 min): Fix the slow query

```
-- Slow: ~5 seconds on 10M row table
SELECT COUNT(*) FROM orders
WHERE EXTRACT(YEAR FROM created_at) = 2024
  AND status = 'completed';
```

Identify the problem and fix it.

Solution:

```
-- Problem: EXTRACT(YEAR FROM created_at) is a function on created_at
--          → index on created_at is not used

-- Fix 1: Rewrite to use range (no function):
SELECT COUNT(*) FROM orders
WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'
  AND status = 'completed';
-- Now can use index on (status, created_at) or (created_at)

-- Fix 2: Create expression index (if rewrite not possible):
CREATE INDEX ON orders(EXTRACT(YEAR FROM created_at), status);
-- Keep original query, now expression index is used
```

Red Flags to Avoid

In an interview, avoid saying:

- "Just add an index to every column" — shows no understanding of write costs
- "Indexes always make queries faster" — ignores selectivity, sequential scans
- "I'd use REINDEX on a production table" — should always say REINDEX CONCURRENTLY
- "Foreign keys are automatically indexed" — PostgreSQL does NOT do this
- "I'd disable autovacuum to improve performance" — terrible idea
- "Hash indexes are faster for everything" — very narrow use case, usually B-tree wins

- "I'd use VACUUM FULL to fix bloat" — exclusive lock, almost never appropriate

---

Key Terminology Cheat Sheet

Term	Definition
Selectivity	Fraction of rows returned by a predicate (0=none, 1=all)
Index Scan	Random access to heap pages via index ctids
Bitmap Index Scan	Collects all ctids, sorts by page, then bulk-reads heap
Index Only Scan	Query answered from index alone (no heap access)
Heap Fetch	Reading a heap page from an index-only scan for visibility check
Index Bloat	Wasted space in index from dead tuples
HOT Update	Update that doesn't touch indexes (non-indexed columns only)
Page Split	Overflow of a full B-tree page, allocating a new page
Fill Factor	% of page fullness at index build time (reserves space for growth)
Posting List	GIN structure: one entry per token mapping to all containing rows
Block Range	BRIN concept: group of N heap pages summarized by min/max
Correlation	Statistical measure of physical vs logical row ordering
Expression Index	Index on a function/expression rather than raw column value
Partial Index	Index covering only rows matching a WHERE predicate
Covering Index	Index containing all columns needed to answer a query
INCLUDE columns	Non-key columns added to index leaf pages for covering scans
Pending List	GIN buffer for deferred merging of new entries (fastupdate)
Visibility Map	Per-page bitmap indicating if all tuples are visible to all transactions
REINDEX CONCURRENTLY	Non-blocking index rebuild (PG12+)
effective_cache_size	Setting telling planner total available cache size
random_page_cost	Cost of a random page read (use 1.1 for SSD, 4.0 default for HDD)